Card Fraud Triage Using a Gaussian Mixture Model Phase 1 Report

H Maheshwari, Vaibhav, Mayank, Keshav, Mitul Bhatia
Team T011, Section A, Project 26
Newton School of Technology, Rishihood University
2401010174, 2401010486, 2401010267, 2401010222, 2401010279

***Abstract*—We build a simple card-fraud review system using a Gaussian Mixture Model (GMM). The model learns patterns from earlier legitimate transactions and assigns larger anomaly scores to less typical later transactions. We compare four component counts using training AIC and BIC, choose alert cutoffs on validation data, and evaluate on a later mixed test period. The selected eight-component GMM achieves trapezoidal precision-recall area 0.1496. Our default validation policy finds 37 of 74 test frauds and falsely alerts on 172 legitimate transactions. Reviewing exactly the top 100 scores finds 29 frauds. The selected GMM improves test PR area over a same-data one-Gaussian reference, whose area is 0.0527. Five ordered notebooks preserve preparation, model settings and validation history. This report covers Phase 1 machine learning only.**

***Index Terms*—fraud triage, Gaussian mixture, anomaly detection, precision-recall**

I. The Problem

Fraud triage means deciding which transactions a person should review first. We cannot inspect every transaction. Our question is: can a model of legitimate activity place fraud near the front of a review queue? Unusual activity is a reason to investigate, not proof of fraud. The course specifies machine learning for Phase 1, so we implement the GMM. The autoencoder remains Phase 2 work.

II. Literature Review

Dempster, Laird and Rubin [1] explain expectation-maximization (EM), which alternates estimating hidden memberships and updating parameters. This explains how our GMM learns its components. Saito and Rehmsmeier [2] explain why precision-recall measures reveal the burden of false alerts when positives are rare. We therefore emphasize PR area, recall, precision and review workload.

The scikit-learn documentation [3] describes GMM settings and information-criterion selection. We use its diagonal-covariance GMM and a small K grid. The recommended data comes from ULB/Worldline [4]. TensorFlow documents the public CSV mirror [5]. These sources explain our design; our result numbers come from our own run.

The limitation connecting these readings is that a well-fitted legitimate density need not separate fraud. EM supplies a fitting method, information criteria select density complexity, and PR measures check whether that density ranking actually supports review. We test this link using later mixed data and fixed validation policies. This is a baseline study, not a new algorithm. The assignment's suggested autoencoder reading by Sakurada and Yairi (2014) is reserved for the later DL phase; we claim no autoencoder results.

TABLE I
Earlier training and later evaluation periods

Set	Transactions	Legitimate	Fraud
Training	170,235	169,893	342
Validation	56,745	56,688	57
Test	56,746	56,672	74

III. Data Quality and Exploration

The original CSV has 284,807 transactions, 31 columns and 492 frauds. Time records elapsed seconds. V1–V28 are anonymous numerical inputs transformed by the provider. Amount is in EUR. Class gives the correct answer: zero for legitimate and one for fraud. We do not know the business meanings of the V columns or how the provider fitted its original PCA.

We verify expected columns, numeric finite values, binary labels and nonnegative Amount/Time. We find no missing or infinite cells. We remove 1,081 exact repeated rows and keep the first occurrence. Nineteen removed rows carry fraud labels, leaving 283,726 transactions and 473 frauds. There is no transaction ID, so duplicate removal is an explicit assumption.

We split earlier and later transactions into approximately 60% training, 20% validation and 20% test. Equal timestamps stay together. Table I shows the counts. The boundaries are 120,393 and 145,234 elapsed seconds. The scaler and GMM use only the 169,893 legitimate training rows. Validation and test retain both classes. Test labels do not choose inputs, K or cutoffs.

Notebook 02 includes six EDA charts: split sizes, training class counts, amount distributions, hourly transaction counts, V1/V2 overlap, and legitimate-feature correlation. Fraud is rare, amounts have a long right tail, and the two classes overlap. A policy predicting all test rows legitimate would have about 99.87% accuracy but catch no fraud. This is why we do not use accuracy as the headline result. Correlation is descriptive: low global correlation does not prove independence inside every mixture component. Plot sampling affects only the V1/V2 picture, not model fitting.

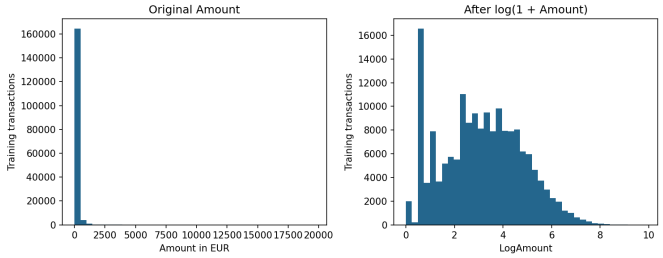


Fig. 1. The log transform compresses the large range of Amount. Both panels use training rows only.

IV. Method, Explained Step by Step

A. Prepare the Inputs

We use 29 inputs: V1–V28 and LogAmount, where

$$\text{LogAmount} = \log(1 + \text{Amount}). \tag{1}$$

The transform accepts zero and compresses large amounts without removing them. We exclude Class because it is the answer. Time determines the split and does not enter the model.

StandardScaler applies $(x - \mu)/\sigma$ to each input, where μ and σ are its mean and standard deviation among legitimate training rows. We fit these numbers once and apply them unchanged to validation and test. Learning scaling values from test data would leak future information into preparation.

B. Learn Legitimate Patterns with a GMM

A Gaussian describes a bell-shaped pattern with a center and spread. A GMM combines K such patterns:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{2}$$

Here π_k is a component's weight, μ_k its center and Σ_k its spread/covariance. The weights sum to one. Components are mathematical patterns, not verified customer categories.

EM repeats two steps. The E-step estimates each transaction's membership in each component. The M-step updates weights, centers and spreads using those estimates. We use diagonal covariance, which learns separate input variances within each component and misses some input relationships. We use three initializations, seed 42, up to 200 iterations, and a small variance addition 10^{-6} to avoid a spread collapsing to zero. All four candidates reach their stopping rule; this does not prove a global optimum.

Labels select known legitimate training rows, but GMM fitting receives only inputs and no target labels. Both facts matter when describing the training setup.

To connect the words to equations, let i identify a transaction, k a component, and j an input. The E-step computes a soft membership, or responsibility:

$$r_{ik} = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{\ell=1}^{K} \pi_\ell \mathcal{N}(x_i \mid \mu_\ell, \Sigma_\ell)}. \tag{3}$$

TABLE II
Training Model Selection; Every Candidate Converged

K	AIC / 10^6	BIC / 10^6	Iterations
1	13.9820	13.9826	2
2	11.9348	11.9360	9
4	10.8944	10.8968	32
8	10.1402	10.1449	35

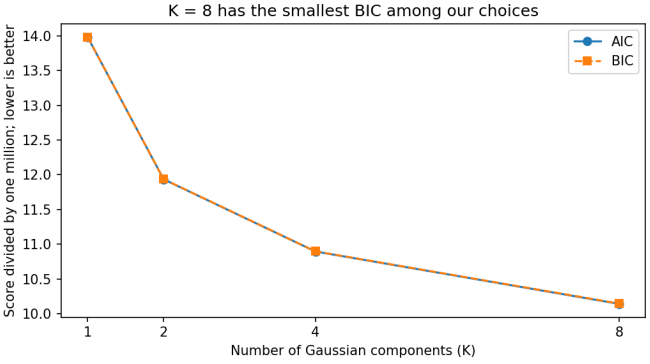


Fig. 2. Eight components give the smallest BIC among our four choices. The curves remain decreasing at the grid boundary.

For one transaction, these memberships sum to one. A transaction can partly influence more than one component. Define $N_k = \sum_i r_{ik}$. The M-step then uses weighted averages:

$$\pi_k = N_k/N, \qquad \mu_k = \frac{1}{N_k} \sum_i r_{ik} x_i, \tag{4}$$

$$\sigma_{kj}^2 = \frac{1}{N_k} \sum_i r_{ik}(x_{ij} - \mu_{kj})^2 + 10^{-6}. \tag{5}$$

The last line learns each input's separate variance for diagonal covariance. The added small value avoids a variance collapsing to zero. These formulas explain what the library fits; the notebook uses the library rather than a second hand-written EM implementation.

C. Choose the Component Count

We try K=1,2,4,8 on the same legitimate rows. AIC and BIC balance a better fit against extra adjustable parameters:

$$\text{AIC} = 2q - 2L, \qquad \text{BIC} = q \log N - 2L. \tag{6}$$

N is the number of fitted rows, q is the number of free parameters, and L is total training log likelihood. Lower values are preferred. We select minimum training BIC and report AIC alongside it. Eight wins within this grid (Fig. 2). It is the largest K tried, so we do not claim it is best among every possible value. Better legitimate density fit does not guarantee better fraud recall. For diagonal covariance, the number of free parameters is $q = K(2d + 1) - 1$, where $d = 29$ inputs. The selected K=8 model therefore has 471 free parameters. More components require more learned numbers, which explains the criterion's complexity penalty.

TABLE III
TEST RESULTS AT FIXED VALIDATION BUDGETS

Val. budget (%)	TP	FP	Recall (%)	Precision (%)	Test FPR (%)
0.1	6	39	8.11	13.33	0.0688
0.5	37	172	50.00	17.70	0.3035
1.0	52	381	70.27	12.01	0.6723

D. Score Transactions and Set Cutoffs

We use anomaly score $s(x) = -\log p(x)$. Low legitimate density produces a larger score. The score is not a probability of fraud. The code uses the negative of `score_samples`; the GMM's `predict` method would instead return component IDs.

We create an alert when the score is strictly above a cutoff. False-positive rate (FPR) means legitimate transactions alerted divided by all legitimate transactions. We predefine validation FPR budgets of 0.1%, 0.5% and 1%. The default 0.5% is a classroom example fixed before test evaluation. Its cutoff is the 99.5th percentile of legitimate validation scores, using the conservative `higher` percentile method. This gives threshold 75.8942. Later test FPR may differ, so we carry this cutoff unchanged into test.

V. RESULTS AND THEIR MEANING

At the default cutoff, the test confusion counts are TN=56,500, FP=172, FN=37 and TP=37. TN means legitimate without an alert; FP means a legitimate false alert; FN means missed fraud; TP means fraud found. Therefore:

$$\text{recall} = \frac{TP}{TP+FN} = \frac{37}{74} = 50\%, \tag{7}$$

$$\text{precision} = \frac{TP}{TP+FP} = \frac{37}{209} = 17.70\%, \tag{8}$$

$$\text{FPR} = \frac{FP}{FP+TN} = \frac{172}{56{,}672} = 0.3035\%. \tag{9}$$

The workload is 209 alerts. We find half the fraud, miss half, and send 172 legitimate transactions for review. These limitations should be stated alongside performance.

Table III applies the three validation cutoffs without retuning on test outcomes. A more permissive policy finds more fraud here, but also creates more false alerts.

The PR curve shows precision against recall across score cutoffs (Fig. 4). Trapezoidal PR AUC connects neighboring points and adds their areas. Its value is 0.1496 on this test period. It is not accuracy or precision at the default cutoff. The test fraud rate is 0.1304%.

The K=1 candidate provides a simple reference: it uses the same legitimate training rows, 29 inputs, scaler and test period, with the same score definition. Its test PR AUC is 0.0527, compared with 0.1496 for the selected GMM. We compare ranking under one area definition, not different evaluation protocols. The selected mixture ranks fraud better in this run; this is not a universal guarantee that adding components improves detection.

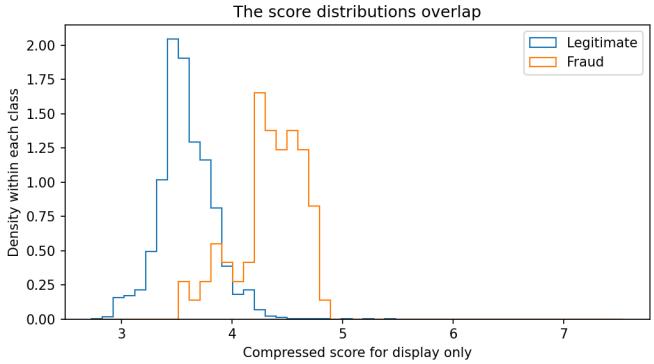


Fig. 3. Final score distributions overlap. The displayed horizontal scale compresses scores only for readability; ranking and cutoff scores remain unchanged.

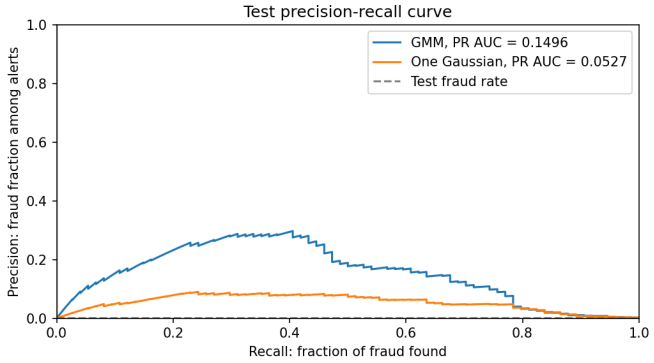


Fig. 4. Final test precision-recall curve. The dashed line marks test fraud prevalence.

TABLE IV
FIXED-CAPACITY REVIEW ON THE SAME TEST BATCH

Reviews	Fraud found	Precision (%)	Recall (%)
50	8	16.0	10.81
100	29	29.0	39.19
200	37	18.5	50.00
500	53	10.6	71.62

Figure 3 explains why a density cutoff can produce both false alerts and missed fraud. Each class has its own density normalization. Negative scores, if any, map to zero only in this displayed transform.

For a fixed batch capacity, sort by decreasing anomaly score and review exactly 100 rows. We find 29 frauds and 71 legitimate transactions, so precision@100 is 29%. Tied scores retain chronological order. This is a capacity for the whole test batch, not a daily workload claim. Table IV gives supporting capacities. Precision need not decrease at every larger capacity because the next ranked block may contain more or fewer frauds.

VI. Reproducibility, Limits and Conclusion

Five ordered notebooks contain the visible analysis and simple explanations: (01) project/literature, (02) quality/EDA, (03) feature engineering, (04) GMM training and (05) evaluation. They exchange saved local files rather than relying on a shared Python session. Read 01, then open 02–05 in order, restarting each kernel and using Run All. All analysis and saving operations appear directly in their cells. FILE_MAP.md links each result and chart to its producing notebook; CODE_GUIDE.md explains the standard functions. The supplied HTML copies show the verified run.

The inline checks cover schema, finite values, labels, time separation, feature exclusion, scaler fitting rows, convergence, empirical validation budgets and consistency of final confusion counts. The run records the input checksum, feature list, model settings, thresholds, software versions and test scores. Python 3.12.4, NumPy 1.26.4, pandas 2.2.2 and scikit-learn 1.4.2 produced the run. Raw data, intermediate arrays and binary model files stay outside Git and are recreated by the notebooks.

The repository provides the literature review, requirement-to-evidence checklist, study guide, demonstration/viva guide, final presentation and editable LaTeX source. AI assistance supported preparation and editing; the team remains responsible for reviewing and understanding the work.

The test period contains only 74 frauds. One fraud decision changes recall by about 1.35 percentage points. The short historical dataset, anonymous inputs, unaudited provider PCA, duplicate-removal assumption, diagonal covariance and small K grid limit the conclusion. Future patterns can change, and rarity is not the same as fraud.

The GMM provides a clear Phase 1 review baseline with visible mistakes and review tradeoffs. We do not claim production readiness. Phase 2 may compare an autoencoder using the same legitimate training rows and evaluation protocol; no autoencoder or hybrid results are included here.

VII. Team Contributions

All five members contributed to the project and jointly wrote this report. Each member contributed the methods, results and explanations for their work. The following division identifies the main project responsibilities. H Maheshwari is referred to as Hardik by the team. Notebook 01 provides the shared project and literature background; the leads below are responsible for their respective stages, while every member prepares to explain the full pipeline.

The report is a joint deliverable of H Maheshwari (Hardik), Vaibhav, Mayank, Keshav and Mitul Bhatia. Table V identifies their main responsibilities; shared report authorship covers the complete methods, results and discussion.

TABLE V
Team T011 contribution responsibilities

Member	Responsibility
H Maheshwari (Hardik) 2401010174	Notebook 05: validation thresholds and evaluation with PR AUC, precision, recall and review capacity.
Vaibhav 2401010486	Notebook 03: feature engineering and training-only scaling.
Mayank 2401010267	Notebook 04: GMM training, EM, AIC/BIC and component selection. README and presentation with Keshav.
Keshav 2401010222	Notebook 02: data quality, chronological splitting and EDA. README and presentation with Mayank.
Mitul Bhatia 2401010279	Overall project lead: planning, managing responsibilities, coordinating all stages and overseeing the project.

References

[1] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. R. Stat. Soc. B*, vol. 39, no. 1, pp. 1–22, 1977, doi: 10.1111/j.2517-6161.1977.tb01600.x.

[2] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[3] scikit-learn, "Gaussian mixture models." [Online]. https://scikit-learn.org/stable/modules/mixture.html.

[4] ULB Machine Learning Group / Worldline, "Credit Card Fraud Detection." [Online]. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud.

[5] TensorFlow, "Classification on imbalanced data," dataset mirror documentation. [Online]. https://www.tensorflow.org/tutorials/structured_data/imbalanced_data. Online sources checked Oct. 21, 2026.